# Exercise 6 — README and User Guide Documentation

## Exercise Description

Use AI to create comprehensive project documentation: a README (Prompt 1), a step-by-step guide for one feature (Prompt 2), and an FAQ (Prompt 3). Project documented: the **Task Manager** (Python).

**Deliverable file:** `README.md` — the generated project README.

---

## Prompt 1 — Project README

See `README.md`. It covers title/description, features, requirements, installation, usage examples for every command, project structure, configuration, troubleshooting, and license — generated from the actual `cli.py` command surface and module layout.

## Prompt 2 — Step-by-Step Guide: "Create your first task and mark it done"

**Prerequisites:** Python 3.8+ installed; a terminal open in the `TaskManager` directory.

1. **Confirm it runs:** `bash python cli.py --help` You should see the list of commands (`create`, `list`, `status`, ...).
2. **Create a task:** `bash python cli.py create "Write report" -d "Q3 summary" -p 3 -u 2026-08-15 -t "work,urgent"` Note the printed `Created task with ID: <id>`.
3. **List tasks** to see it and copy the id: `bash python cli.py list`
4. **Mark it done:** `bash python cli.py status <id> done` → `Updated task status to done`.
5. **Verify with stats:** `bash python cli.py stats` The "done" count and "Completed in last 7 days" should reflect your task.

**Common mistakes:** using a truncated id from the list display (use the full id); wrong date format (must be `YYYY-MM-DD`); forgetting quotes around a multi-word title.

## Prompt 3 — FAQ

**Q: Do I need to install anything besides Python?** A: No — it uses only the standard library.

**Q: Where are my tasks stored?** A: In `tasks.json` in the directory you run the command from. It's created on first write.

**Q: How do priorities work?** A: `1`=LOW, `2`=MEDIUM (default), `3`=HIGH, `4`=URGENT. Filter with `list -p N`.

**Q: How do I see only overdue tasks?** A: `python cli.py list -o`. A task is overdue if its due date has passed and it isn't done.

**Q: Why does the id in `list` look shorter than the one I created?** A: The list view truncates the id for readability; commands still expect the full id.

**Q: Can two people share the same `tasks.json`?** A: Not safely — writes rewrite the whole file (last-writer-wins). Use separate files or a real datastore for multi-user.

---

## Submission for Exercise 6 — README and User Guide Documentation

**1. Project information chosen:** the Python Task Manager (CLI, standard library only).

**2. Comprehensive README (Prompt 1):** delivered as `README.md` — features, install, per-command usage, structure, config, troubleshooting, license.

**3. Step-by-step guide (Prompt 2):** "create your first task and mark it done", with prerequisites, five numbered steps, verification, and common mistakes.

**4. FAQ (Prompt 3):** six Q&As covering dependencies, storage, priorities, overdue filtering, id truncation, and multi-user safety.

**5. Notes:** the most valuable move was generating the README **from the real `cli.py` argument definitions** rather than a generic template — it kept the flags/examples accurate. Structure lesson: README = reference, step-by-step guide = one happy path, FAQ = the sharp edges (id truncation, last-writer-wins) that a reference section tends to bury.